

Speculative Decoding, Worked Out by Hand

HOW A CHEAP DRAFT MODEL EMITS MANY TOKENS PER EXPENSIVE FORWARD PASS

What this document does. Batching (Topic 1) raised *throughput* but did nothing for single-request *latency*: every request still waits one big-model forward pass per token. Speculative decoding breaks that floor. A small *draft* model proposes K tokens; the big *target* model verifies all K in a single forward pass and emits as many as it accepts. This document derives the central number — expected accepted tokens per iteration — as a geometric series, then computes the resulting effective TPOT and speedup for $\alpha = 0.7$, $K = 4$. Every figure is reproduced by `m05_t02_speculative_decoding_engine.py` (Appendix A).

The one idea. One big-model forward pass already computes logits at *every* position. Pre-extend the sequence with K guessed tokens and that single pass tells you what the big model *would* have said at each — so you can accept a whole run of correct guesses for the price of one pass.

CONCEPTS COVERED, IN ORDER

the draft-and-verify cycle; why acceptance is independent per position; the expected-token geometric series $E[\text{tok}] = 1 + \alpha + \dots + \alpha^K$ (derived); its closed form; iteration time; effective TPOT; speedup; the α - and K -sensitivity sweeps; and the optimal K .

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The system under study	2
2	Step 1 — The draft-and-verify cycle	2
3	Step 2 — Deriving the expected tokens per iteration	2
4	Step 3 — The worked example: $\alpha = 0.7$, $K = 4$	3
5	Step 4 — Sensitivity to α	4
6	Step 5 — Sensitivity to K , and the optimal length	4
7	The argument, at a glance	5
7.1	Equation cheat-sheet	5
	Appendix A — Reference implementation	5

1 The system under study

A 1–2B draft model shadows the 70B *Nexus* target. The draft runs K fast serial passes; the target runs one verification pass over all K .

Quantity	Symbol	Value here
Per-token acceptance prob.	α	0.70
Speculation length	K	4
Draft forward pass	t_{draft}	1.2 ms
Target forward pass (baseline TPOT)	t_{target}	12.0 ms

α is the probability that, at a given position, the target accepts the draft’s proposed token. It captures how well the small model mimics the big one on this workload.

Intuition

Think of the draft as an eager intern writing the next few words and the target as the senior reviewer. The reviewer reads the whole proposed run in one sitting (one forward pass), strikes through at the first word it disagrees with, and keeps everything before. If the intern is right 70% of the time, accepted runs tend to be short — but even a short accepted run means several tokens emitted for the cost of *one* review.

2 Step 1 — The draft-and-verify cycle

One iteration of speculative decoding:

DRAFT • cheap model, K passes

Run K serial draft passes (< 1 ms each).

Emit $d_1, d_2, \dots, d_K$ from $q(\cdot)$.

Cost: $K \cdot t_{\text{draft}}$.

VERIFY • big model, 1 pass

Target sees context + $d_1 \dots d_K$ at once.

Emits target logits $p(\cdot)$ at all $K+1$ positions.

Accept the longest agreeing prefix; emit +1 free token.

Cost: t_{target} .

Accept token d_i with probability $\min(1, p(d_i)/q(d_i))$. The first rejection breaks the chain — everything after it is discarded — but the target then contributes one *guaranteed* token at that position (resampled losslessly; Topic 3 proves this keeps the distribution exactly p). So a run of j accepted draft tokens yields $j + 1$ emitted tokens.

The payoff structure

If the first j draft tokens are accepted and the $(j+1)^{\text{th}}$ is rejected (or $j = K$), the iteration emits $j + 1$ tokens for the cost of *one* target pass plus K cheap draft passes. The win is entirely about how long the accepted run tends to be — which is governed by α .

3 Step 2 — Deriving the expected tokens per iteration

Acceptances are independent across positions, each with probability α . We want the expected number of emitted tokens per iteration.

The geometric series, step by step

Let N be the number of tokens emitted in one iteration. The first j draft tokens are accepted (prob. α^j for the run reaching length j), and emission always adds the one guaranteed target token. The cleanest way to count $E[N]$ is by indicator: position i ($i = 0, 1, \dots, K$) contributes a token iff all i draft tokens before it were accepted. Position 0 (the guaranteed target token) always fires; position $i \geq 1$ fires with probability α^i .

$$E[N] = \sum_{i=0}^K \Pr[\text{first } i \text{ accepted}] = \sum_{i=0}^K \alpha^i = 1 + \alpha + \alpha^2 + \dots + \alpha^K.$$

This is a finite geometric series. Multiply by $(1 - \alpha)$ and telescope:

$$(1 - \alpha) \sum_{i=0}^K \alpha^i = \sum_{i=0}^K \alpha^i - \sum_{i=0}^K \alpha^{i+1} = 1 - \alpha^{K+1},$$

so, for $\alpha < 1$,

$$E[\text{tokens}] = \frac{1 - \alpha^{K+1}}{1 - \alpha} = 1 + \alpha + \dots + \alpha^K.$$

Expected tokens per speculation iteration

$$E[\text{tokens}] = \frac{1 - \alpha^{K+1}}{1 - \alpha} \quad (\alpha < 1), \quad t_{\text{iter}} = K t_{\text{draft}} + t_{\text{target}},$$

$$\text{eff-TPOT} = \frac{t_{\text{iter}}}{E[\text{tokens}]}, \quad \text{speedup} = \frac{t_{\text{target}}}{\text{eff-TPOT}}.$$

Intuition

The series $1 + \alpha + \dots + \alpha^K$ has an honest reading: the "1" is the token the target always emits; the " α " is the chance you also keep the first draft token; the " α^2 " the chance you keep the second as well; and so on. Each further term is smaller because keeping the i^{th} token requires *all* i before it to have survived. The geometry is exactly the compounding survival of an accepted run.

4 Step 3 — The worked example: $\alpha = 0.7$, $K = 4$ **Expected tokens, term by term**

$$\begin{aligned} \alpha^0 &= 1.00000 \\ \alpha^1 &= 0.70000 \\ \alpha^2 &= 0.49000 \\ \alpha^3 &= 0.34300 \\ \alpha^4 &= 0.24010 \end{aligned}$$

$$E[\text{tokens}] = 1 + 0.7 + 0.49 + 0.343 + 0.2401 = \mathbf{2.7731}.$$

Closed-form cross-check:

$$E[\text{tokens}] = \frac{1 - 0.7^5}{1 - 0.7} = \frac{1 - 0.16807}{0.3} = \frac{0.83193}{0.3} = 2.7731. \quad \checkmark$$

Iteration time, effective TPOT, speedup

$$\begin{aligned}
t_{\text{iter}} &= K t_{\text{draft}} + t_{\text{target}} = 4 \times 1.2 + 12.0 = 4.8 + 12.0 = 16.80 \text{ ms}, \\
\text{eff-TPOT} &= \frac{t_{\text{iter}}}{E[\text{tokens}]} = \frac{16.80}{2.7731} = 6.058 \text{ ms per token}, \\
\text{speedup} &= \frac{t_{\text{target}}}{\text{eff-TPOT}} = \frac{12.0}{6.058} = \mathbf{1.98\times}. \quad \checkmark
\end{aligned}$$

The baseline emitted one token every 12 ms. Speculation emits one every 6.06 ms — a $1.98\times$ speedup, a 49.5% cut in per-token cost — while producing *exactly* the target model’s distribution (Topic 3).

Mini-example — where the 4.8 ms of draft cost goes

The iteration spends 12 ms on the one target pass and 4.8 ms on four draft passes — 29% overhead. That overhead buys 2.77 tokens instead of 1. As long as $E[\text{tokens}] > t_{\text{iter}}/t_{\text{target}} = 16.8/12 = 1.40$, speculation wins. Here $2.77 \gg 1.40$, so it wins comfortably.

5 Step 4 — Sensitivity to α

Acceptance rate is the dominant lever. Holding $K = 4$:

α	$E[\text{tokens}]$	speedup
0.50	1.938	1.384×
0.60	2.306	1.647×
0.70	2.773	1.981×
0.80	3.362	2.401×
0.90	4.095	2.925×

Each 0.10 of acceptance is worth roughly a quarter-turn of speedup. A draft model that “drifts” — its distribution slowly diverging from the target as the workload changes — silently bleeds speedup. This is why production systems *monitor* α as a first-class metric.

Watch out

α is a property of the *draft-target-workload triple*, not of the algorithm. A draft tuned on last month’s traffic can decay from $\alpha = 0.8$ to 0.6 as prompts shift, dropping speedup from $2.40\times$ to $1.65\times$ — a 31% throughput regression with no code change and no error in the logs. Track α continuously; alarm when it falls.

6 Step 5 — Sensitivity to K , and the optimal length

More speculation is not always better. Each extra draft token adds t_{draft} to *every* iteration, but only rarely survives to be emitted (its survival probability is α^K , which shrinks fast). Holding $\alpha = 0.70$:

K	E[tokens]	t_{iter} (ms)	speedup
1	1.700	13.20	1.545×
2	2.190	14.40	1.825×
4	2.773	16.80	1.981×
6	3.059	19.20	1.912×
8	3.199	21.60	1.777×

The speedup rises, peaks, and *falls*. At $\alpha = 0.70$ the optimum is $K = 4$ (speedup 1.981×); beyond it, the linear $K t_{\text{draft}}$ cost outruns the saturating E[tokens] gain.

Tune K to α

The best K grows with α : a more accurate draft can profitably speculate further because its tokens survive longer. At $\alpha = 0.70$, $K = 4$ is optimal; at $\alpha = 0.90$ the optimum is larger. There is no universal K — it is a function of how good the draft is and the draft/target cost ratio $t_{\text{draft}}/t_{\text{target}}$.

7 The argument, at a glance

#	Step	Result
1	Cycle	draft K tokens; verify in 1 pass; emit accepted run +1
2	Derive	$E[\text{tok}] = \sum_{i=0}^K \alpha^i = (1 - \alpha^{K+1})/(1 - \alpha)$
3	Worked	$\alpha=0.7, K=4 \Rightarrow E=2.773$, speedup = 1.98× ✓
4	α sweep	0.5→0.9: speedup 1.38 × →2.93×
5	K sweep	optimum $K=4$ at $\alpha=0.7$; over-speculation hurts

7.1 Equation cheat-sheet

Geometric series	$1 + \alpha + \dots + \alpha^K = \frac{1 - \alpha^{K+1}}{1 - \alpha}$
Expected tokens	$E[\text{tokens}] = \frac{1 - \alpha^{K+1}}{1 - \alpha}$
Iteration time	$t_{\text{iter}} = K t_{\text{draft}} + t_{\text{target}}$
Effective TPOT	$\text{eff-TPOT} = t_{\text{iter}}/E[\text{tokens}]$
Speedup	$t_{\text{target}}/\text{eff-TPOT}$
Break-even	$E[\text{tokens}] > t_{\text{iter}}/t_{\text{target}}$

Appendix A — Reference implementation

Every number above is produced by `m05_t02_speculative_decoding_engine.py`. It verifies the series term-by-term against the closed form and runs α - and K -sensitivity sweeps.

```

1 """
2 Reference implementation for: Module 05 - Topic 2
3 "Speculative Decoding - many tokens per big-model forward pass."
4
5 Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6 so the arithmetic in the document is exact and self-consistent.
7 """
8

```

```

9 # -----
10 # 0. The setup: a 1B draft proposes K tokens, the 70B target verifies.
11 # -----
12 alpha    = 0.70    # per-token acceptance probability (draft agrees with target)
13 K        = 4       # speculation length (draft tokens per iteration)
14 draft_ms = 1.2     # one draft-model forward pass (ms)
15 target_ms = 12.0   # one target-model forward pass (ms) -- the baseline TPOT
16
17 # -----
18 # 1. Expected accepted tokens per iteration: geometric series.
19 #    $E[\text{tokens}] = 1 + a + a^2 + \dots + a^K = (1 - a^{K+1}) / (1 - a)$ 
20 #   (the +1 token beyond K accepted is the target's own free token /
21 #   the resample on first rejection; the series  $1 + \dots + a^K$  counts the
22 #   expected length of the accepted run plus the guaranteed bonus token.)
23 # -----
24 series = sum(alpha**i for i in range(K + 1))    #  $1 + a + \dots + a^K$ 
25 closed = (1 - alpha**(K + 1)) / (1 - alpha)    # closed form
26 print("== 1. Expected tokens per iteration ==")
27 print(f"alpha = {alpha}, K = {K}")
28 for i in range(K + 1):
29     print(f"alpha^{i} = {alpha**i:.5f}")
30 print(f"sum 1+a+...+a^K (term by term) = {series:.5f}")
31 print(f"closed form (1-a^(K+1))/(1-a) = {closed:.5f}")
32 print(f"match: {abs(series-closed) < 1e-12}")
33
34 E_tokens = closed
35
36 # -----
37 # 2. Iteration time, effective TPOT, speedup.
38 # -----
39 iter_ms = K * draft_ms + target_ms
40 eff_tpot = iter_ms / E_tokens
41 speedup = target_ms / eff_tpot
42 print("\n== 2. Timing and speedup ==")
43 print(f"iter_time    = K*draft + target = {K}*{draft_ms} + {target_ms} = {iter_ms:.2f} ms")
44 print(f"E[tokens]     = {E_tokens:.4f}")
45 print(f"eff_TPOT      = iter_time / E[tokens] = {iter_ms:.2f} / {E_tokens:.4f} = {eff_tpot:.4f} ms")
46 print(f"speedup       = target_ms / eff_TPOT = {target_ms} / {eff_tpot:.4f} = {speedup:.3f}x")
47 print(f"per-token cost reduction = {100*(1 - eff_tpot/target_ms):.1f}%")
48
49 # -----
50 # 3. Sensitivity sweep over alpha (fixed K) and over K (fixed alpha).
51 # -----
52 def speedup_of(a, k):
53     et = (1 - a**(k + 1)) / (1 - a)
54     it = k * draft_ms + target_ms
55     return target_ms / (it / et), et
56
57 print("\n== 3a. alpha sweep (K=4) ==")
58 print(f"{'alpha':>7} {'E[tok]':>8} {'speedup':>9}")
59 for a in [0.50, 0.60, 0.70, 0.80, 0.90]:
60     sp, et = speedup_of(a, K)
61     print(f"{a:>7.2f} {et:>8.3f} {sp:>8.3f}x")
62
63 print("\n== 3b. K sweep (alpha=0.70) ==")
64 print(f"{'K':>3} {'E[tok]':>8} {'iter_ms':>9} {'speedup':>9}")
65 for k in [1, 2, 4, 6, 8]:
66     sp, et = speedup_of(alpha, k)
67     it = k * draft_ms + target_ms
68     print(f"{k:>3} {et:>8.3f} {it:>9.2f} {sp:>8.3f}x")
69
70 # -----
71 # 4. Diminishing returns: best K for alpha=0.70.

```

```
72 # -----
73 best_k, best_sp = max((k, speedup_of(alpha, k)[0]) for k in range(1, 21)),
74                     key=lambda t: t[1])
75 print("\n== 4. Optimal K for alpha=0.70 ==")
76 print(f"best K = {best_k} -> speedup = {best_sp:.3f}x")
77 print("Beyond the optimum, extra draft passes cost more than the rare extra accept.")
```

— END OF DOCUMENT —